

VM Setup Specs

Operating System: Ubuntu 22.04 LTS x86/64

Storage: 150GB

Machine Type: g2-standard-8 (8 vCPUs, 32 GB memory)

GPU: 1x NVIDIA L4

NOTE: Using `realpath <filename>` would help for downloading files from VM to local

Setting up Conda and running Shell Scripts

From root directory...

```
#####
```

Installing Miniconda

```
#####
```

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

```
# ENTER > yes > ENTER > yes
```

```
bash ~/Miniconda3-latest-Linux-x86_64.sh
```

```
# CLOSE THE SSH INSTANCE, AND REOPEN IT
```

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
```

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

```
#####
```

Cloning Repository

```
#####
```

```
git clone https://github.com/chengharv-pen/comp433_braintotext.git
```

```
cd comp433_braintotext/
```

```
#####
```

Shell Scripts

```
#####
```

```
bash setup.sh
```

```
# Sanity check
```

```
conda activate b2txt25
```

```
sudo apt-get install build-essential
```

```
sudo apt-get install cmake
```

```
# fix for faulty setup_lm shell script?????
```

```
sudo apt-get install -y zlib1g-dev
```

```
# THIS ONE TAKES A WHILE
```

```
bash setup_lm.sh
```

```
# Sanity check
```

```
conda activate b2txt25_lm
```

#####

Installing Redis

#####

one-liner

```
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/  
redis-archive-keyring.gpg
```

one-liner

```
echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] https://packages.  
redis.io/deb $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/redis.list
```

```
sudo apt-get update
```

```
sudo apt-get install redis
```

Making PyTorch recognize NVIDIA GPUs

```
sudo apt update
```

```
sudo apt install -y nvidia-cuda-toolkit
```

```
sudo apt install -y ubuntu-drivers-common
```

```
sudo ubuntu-drivers install
```

```
sudo reboot
```

```
# CLOSE THE SSH INSTANCE, AND REOPEN IT
```

```
# Whenever you SSH back into the GCP VM:  
nvidia-smi
```

```
conda activate b2txt25
```

Downloading the Dryad Dataset

```
cd comp433_braintotext/
```

```
python download_data.py
```

Training the Model

```
cd comp433_braintotext/project_files/<model-type>
```

```
conda activate b2txt25
```

```
python train_model.py
```

```
# if you somehow get an error with this, try:  
rm -r trained_models
```

```
# OR modify the path appropriately in the args YAML file
```

Evaluating the Model

```
cd comp433_braintotext
```

```
# Start redis server
sudo sysctl vm.overcommit_memory=1
redis-server --daemonize yes
```

```
# Sanity check
redis-cli
ping # output should be PONG
# Ctrl+C out of redis-cli
```

```
# Start language model
conda activate b2txt25_lm
```

```
# the & at the end is really important,
# need it to keep running as a background process
# this is a one-liner
```

```
python language_model/language-model-standalone.py --lm_path language_model/
pretrained_language_models/openwebtext_1gram_lm_sil --do_opt --nbest 100 --
acoustic_scale 0.325 --blank_penalty 90 --alpha 0.55 --redis_ip localhost --
gpu_number 0 &
```

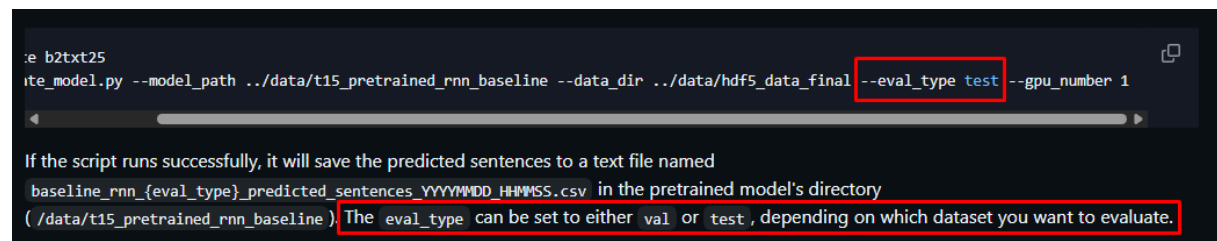
```
# you may need to press Enter after its done
```

```
# Evaluate
conda activate b2txt25
```

```
cd project_files/cnn_transformer
```

```
# Runs the pretrained baseline RNN model
# Change --model_path to the appropriate directory if using a newly-trained model
# THIS TAKES A WHILE TO RUN, BUT MUCH SHORTER THAN TRAIN (~10min)
# this is a one-liner
```

```
python evaluate_model.py --model_path trained_models/cnn_transformer_attempt1 --
data_dir ../../data/hdf5_data_final --eval_type val --gpu_number 0
```



```
conda activate b2txt25
python evaluate_model.py --model_path ../data/t15_pretrained_rnn_baseline --data_dir ../data/hdf5_data_final --eval_type test --gpu_number 1
```

If the script runs successfully, it will save the predicted sentences to a text file named `baseline_rnn_{eval_type}_predicted_sentences_YYYYMMDD_HHMMSS.csv` in the pretrained model's directory (`../data/t15_pretrained_rnn_baseline`). The `eval_type` can be set to either `val` or `test`, depending on which dataset you want to evaluate.

```
# tracking and killing language model [IF NEEDED]
tail -f opt_model.log
ps aux | grep language-model-standalone.py
kill <PID>
```

(OPTIONAL) Accessing remote repository from the GCP VM

Generate SSH key in Ubuntu VM

```
ssh-keygen -t ed25519 -C "<your-github-email>"
```

Press enter twice

Run this...

```
cat ~/.ssh/id_ed25519.pub
```

You will see something like:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE... <your-github-email>
```

Then go to:

GitHub → Settings → SSH and GPG keys → New SSH key

- Title: Ubuntu VM
- Key type: Authentication
- Key: (paste the copied line)
- Click Add SSH key

Go to repo folder in Ubuntu VM

```
cd ~/comp433_braintotext
```

```
# this is a one liner
```

```
git remote set-url origin git@github.com:<your-github-username>/  
comp433_braintotext.git
```

```
git remote -v
```

Test SSH connection

```
ssh -T git@github.com
```

Set configurations

```
git config --global user.email "<your-github-email>"
```

```
git config --global user.name "<your-github-username>"
```

Try this (if you want to)

```
git pull
```

```
# do some stuff
```

```
git add .
```

```
git status
```

```
git commit -m "test commit from VM"
```

```
git push
```